

SQL PROJECT: Online Bookstore

```
--create Database
CREATE DATABASE Online_Bookstore;

--create tables
DROP TABLE IF EXISTS Books;
CREATE TABLE Books(
Book_ID SERIAL PRIMARY KEY,
Title VARCHAR(100),
Author VARCHAR(100),
Genre VARCHAR(50),
Publiished_Year INT,
Price NUMERIC(10,2),
Stock INT
);

DROP TABLE IF EXISTS customers;
CREATE TABLE Customers(
Customer_ID SERIAL PRIMARY KEY,
Name VARCHAR(100),
Email VARCHAR(100),
Phone VARCHAR(15),
City VARCHAR(50),
Country VARCHAR(150)
);

DROP TABLE IF EXISTS orders;
CREATE TABLE Orders(
Order_ID SERIAL PRIMARY KEY,
Customer_ID INT REFERENCES Customers(Customer_ID),
Book_ID INT REFERENCES Books(Book_ID),
Order_Date DATE,
Quantity INT,
Total_Amount NUMERIC(10,2)
);

SELECT * FROM Books;
SELECT * FROM Customers;
SELECT * FROM Orders;

--Import Data Into Books Table
COPY Books(Book_ID,Title,Author,Genre,Publiished_Year,Price,Stock)
FROM 'C:\Users\HP\Downloads\Books.csv'
CSV HEADER;

--Import Data into Customers Table
```

```
COPY Customers (Customer_ID,Name,Email,Phone,City,Country)
FROM 'C:\Users\HP\Downloads\Customers.csv'
CSV HEADER;
```

```
--Import Data into Orders Table
COPY
Orders (Order_ID,Customer_ID,Book_ID,Order_Date,Quantity,Total_Amount)
FROM 'C:\Users\HP\Downloads\Orders.csv'
CSV HEADER;
```

1) Retrieve all books in the "Fiction" genre:

```
SELECT * FROM Books
WHERE Genre = 'Fiction';
```

2) Find books published after the year 1950:

```
SELECT * FROM Books
WHERE Published_Year>1950;
```

3) List all customers from the Canada:

```
SELECT * FROM Customers
WHERE Country='Canada';
```

4) Show orders placed in November 2023;

```
SELECT * FROM Orders
WHERE order_date BETWEEN '2023-11-01' AND '2023-11-30' ;
```

5) Retrieve the total stock of books available:

```
SELECT SUM(Stock) as Total_stock
FROM Books;
```

6) Find the details of most expensive books:

```
SELECT * FROM Books
ORDER BY Price DESC
LIMIT 1;
```

7) Show all customers who ordered more than 1 quantity of a book:

```
SELECT * FROM Orders
WHERE quantity>1;
```

8) Retrieve all orders where the total amount exceeds \$20:

```
SELECT * FROM Orders
WHERE total_amount>20;
```

9) List all genres available in the books table:

```
SELECT DISTINCT Genre
FROM Books;
```

10) Find the book with the lowest stock:

```
SELECT * FROM Books ORDER BY Stock
LIMIT 1;
```

11) Calculate the total revenue generated from all orders:

```
SELECT SUM(total_amount) AS Revenue
```

FROM Orders;

12) Retrieve the total number of books sold for each genre:
SELECT b.Genre,SUM(o.Quantity) AS Total_Books_sold
FROM Orders o
JOIN Books b ON o.book_id = b.book_id
GROUP BY b.Genre;

13) Find the average price of books in the 'Fantasy' genre:
SELECT AVG(Price) AS Average_price
FROM Books
WHERE Genre = 'Fantasy';

14) List customers who have placed at least 2 orders:
SELECT o.customer_id,c.name, COUNT(o.Order_id) AS Order_count
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
GROUP BY o.Customer_id , c.name
HAVING COUNT(order_id) >=2;

15) Find the most Frequently ordered book:
SELECT o.Book_id,b.title,COUNT(o.order_id) AS Order_count
FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY o.book_id,b.title
ORDER BY Order_count DESC LIMIT 1;

16) Show the top 3 most expensive books of 'Fantasy' Genre:
SELECT * FROM Books
WHERE genre = 'Fantasy'
ORDER BY Price DESC LIMIT 3;

--17) Retrieve the total quantity of books sold by each author:
SELECT b.author, SUM(o.quantity) AS Total_books_sold
FROM orders o
JOIN books b ON o.book_id = b.book_id
GROUP BY b.Author;

18) List the cities where customers who spent over \$30 are located:
SELECT DISTINCT c.city, total_amount
FROM orders o
JOIN customers c ON o.customer_id= c.customer_id
WHERE o.total_amount > 30;

19) Find the customers who spent the most on orders:
SELECT c.customer_id, c.name,SUM(o.total_amount) AS Total_spent
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id , c.name
ORDER BY total_spent DESC LIMIT 1;

20) Calculate the stock remaining after fulfilling all orders:
SELECT b.Book_id,b.title,b.stock, COALESCE(SUM(o.quantity),0) AS
order_quantity,
b.stock - COALESCE(SUM(o.quantity),0) as remaining_quantity
FROM Books b
LEFT JOIN orders o ON b.Book_id = o.Book_id

```
GROUP BY b.book_id ORDER BY b.book_id;
```